Orchestration for efficient LLM Checkpointing within HPC

Advanced Computing Project 25/26

André Miranda	PG60238
Gustavo Barros	PG61527
José Soares	PG60277
Miguel Carvalho	PG61153

Department of Informatics – School of Engineering – University of Minho

April 24st 2026

Table of contents

Context

Training Large Language Models (LLMs) involves refining billions of parameters over several weeks of continuous computation.

To ensure resilience against hardware failures and prevent total progress loss, model states must be periodically saved to a Parallel File System (PFS).

Problem: The Checkpointing Overhead

Challenge	Impact
PFS Contention	Simultaneous writes from multiple nodes saturate the shared storage bandwidth.
Training Stalls	Heavy I/O operations block the training process, leading to significant idling.
Noisy Neighbor	Large-scale checkpointing bursts degrade performance for all concurrent HPC jobs.

Table 1: Technical bottlenecks of direct checkpointing to PFS.

Motivation

The goal is to design a checkpointing system that:

- Mitigates I/O contention on the PFS, ensuring fair resource sharing and Cluster Harmony.
- Provides resilience against hardware failures, minimizing training disruptions.
- Ensures QoS policies for checkpoint reliability with overall cluster performance.

Approach: Policy-Driven Checkpointing

Tiered Persistence

Local Storage as a buffer, decoupling training from PFS latency.

Controlled PFS Access

Mitigate I/O contention via a centralized **Orchestrator**.

Policy-Driven

Dynamic migration based on cluster-wide scheduling policies.

Traffic Shaping

Token Bucket to smooth traffic and eliminate noisy neighbors.

Architecture

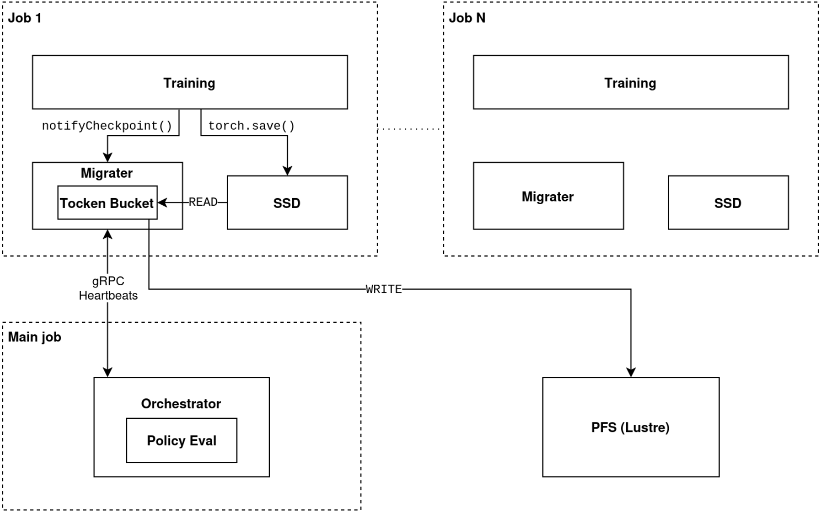

Figure 1. System Architecture.

Current Work

- **End-to-end checkpointing pipeline**:
  - Checkpoints saved locally with asynchronous support for `torch.save`.
  - Fallback to direct PFS copy if Migrater notification fails.
  - Background migration to PFS.
- **gRPC-based coordination layer**:
  - PyTorch → Migrater: notify checkpoint completion + timestamp.
  - Migrater → Orchestrator: checkpoint metadata + node state.
  - Orchestrator → Migrater: `START_FLUSH` / `CHANGE_RATE` / `HOLD`.

Current Work (Cont.)

- **Policy-driven scheduling**:
  - **Fixed-Rate**: same bandwidth cap for every worker.
  - **Uniform Fair-Share**: equal split across all registered workers.
  - **Active Fair-Share**: equal split across workers with pending data only.

Next Steps: Scaling and Benchmarking

Deucalion Deployment

Transition from local testing to the **Deucalion supercomputer** to validate scalability across multiple ARM/X86 nodes.

Policy Refinement

Evaluate different **scheduling policies** within the Orchestrator to find the optimal balance for PFS throughput.

Profiling

Extract performance data using the **PyTorch Profiler** during large-scale runs (already configured).

System Robustness

Ensure the Token Bucket performs reliably during massive concurrent transfers, preventing any bypass of the defined I/O policies.

Orchestration for efficient LLM Checkpointing within HPC

Advanced Computing Project 25/26

André Miranda	PG60238
Gustavo Barros	PG61527
José Soares	PG60277
Miguel Carvalho	PG61153

Department of Informatics – School of Engineering – University of Minho

April 24st 2026